
GANPAT UNIVERSITY

Faculty of Engineering & Technology
Institute of Computer Technology

GridLogic

A Data-Driven Formula 1 Race Prediction & Simulation System

Subject	Predictive Analysis (2CSE-608)
Programme	B.Tech in Computer Science & Engineering
Year / Semester	3rd Year — Semester VI
Academic Year	2025 – 2026
University	Ganpat University, Mehsana, Gujarat — 384012
Submitted To	Institute of Computer Technology

Submitted in partial fulfilment of the requirements of the subject

Predictive Analysis (2CSE-608)

DECLARATION

We hereby declare that the project report titled "GridLogic: A Data-Driven Formula 1 Race Prediction and Simulation System" submitted to Ganpat University in partial fulfilment of the requirements of the subject Predictive Analysis (2CSE-608), B.Tech in Computer Science & Engineering, Semester VI, Academic Year 2025–26, is a record of original work carried out by us.

The work presented in this report has not been submitted previously for any other examination, degree, or diploma at this or any other institution. All sources of information and assistance have been duly acknowledged.

Student Signature: _____ Date: _____

Guide / Faculty Signature: _____ Date: _____

CERTIFICATE

This is to certify that the project titled "GridLogic: A Data-Driven Formula 1 Race Prediction and Simulation System" has been carried out by the student(s) of B.Tech in Computer Science & Engineering, 3rd Year, Semester VI, Ganpat University, under my supervision, in partial fulfilment of the requirements of the subject Predictive Analysis (2CSE-608) for the Academic Year 2025–26.

The project demonstrates satisfactory understanding and application of predictive analytics concepts including supervised regression, feature engineering, hyperparameter tuning, model evaluation, and data visualization.

Guide / Subject Faculty

Institute of Computer Technology
Ganpat University, Mehsana, Gujarat

ACKNOWLEDGEMENT

We express our sincere gratitude to the subject faculty of Predictive Analysis (2CSE-608) at Ganpat University for their continuous guidance, support, and constructive feedback throughout the development of this project.

We are thankful to the Institute of Computer Technology, Ganpat University, for providing the academic environment and computational resources necessary to complete this project.

We acknowledge the developers of the FastF1 open-source Python library (Tim Volk and contributors), without which access to real Formula 1 timing and telemetry data would not have been possible for a student-level project.

We also acknowledge the contributions of the XGBoost, scikit-learn, pandas, numpy, and matplotlib open-source communities whose tools form the technical backbone of this system.

The Student(s)

B.Tech CSE — 3rd Year, Semester VI

Ganpat University, 2025–26

ABSTRACT

Subject: Predictive Analysis (2CSE-608) | Ganpat University | B.Tech CSE 3rd Year | 2025–26

Formula 1 is among the most data-intensive competitive sports in the world. Each Grand Prix weekend generates millions of structured telemetry data points including sector timings, tyre temperatures, fuel loads, speed trap readings, DRS activations, and GPS coordinates. This provides an exceptionally rich dataset for predictive modelling. Despite this data richness, accurately predicting race outcomes in a lap-by-lap manner using open-source tools and student-accessible hardware remains a challenging and largely unsolved problem in the public domain.

This project, GridLogic, addresses this challenge through a complete end-to-end predictive analytics pipeline built entirely in Python. The system fetches real Formula 1 race data through the FastF1 open-source API, engineers a domain-informed feature set from lap-level telemetry, trains an XGBoost gradient boosting regression model to predict individual lap times, and channels those predictions into a deterministic lap-by-lap race simulation engine. The simulation output is rendered as a real-time animated visualization using matplotlib FuncAnimation, displaying all twenty drivers moving along a parametric circuit layout with a dynamically updating leaderboard and accurate gap calculations.

The project directly applies core Predictive Analysis subject topics: supervised regression modelling, feature engineering and selection, hyperparameter optimization via GridSearchCV, train/test evaluation with MAE and RMSE metrics, and feature importance interpretation. The XGBoost model achieves a Mean Absolute Error of 0.67 seconds on lap time prediction which is sub-second accuracy that produces realistic and verifiable race outcome predictions.

Trained on multi-year historical data from 2019, 2020, 2021, 2022, and 2023 for the Bahrain Grand Prix, the system correctly predicts the exact podium finishing order — Verstappen, Perez, Alonso — matching the actual 2023 race result. Eight of the top ten positions are within two places of the actual classification. The complete pipeline runs from user input to animated race visualization in under five minutes on standard student hardware.

Keywords: FastF1, XGBoost, Gradient Boosting, Feature Engineering, Predictive Analytics, Supervised Regression, Lap Time Prediction, Tyre Degradation, Race Simulation, GridSearchCV, matplotlib, FuncAnimation, Python, Formula 1

TABLE OF CONTENTS

DECLARATION.....	2
CERTIFICATE.....	3
ACKNOWLEDGEMENT.....	4
ABSTRACT.....	5
TABLE OF CONTENTS.....	6
LIST OF FIGURES.....	9
LIST OF TABLES.....	9
LIST OF ABBREVIATIONS.....	10
CHAPTER 1 — INTRODUCTION.....	11
1.1 Overview of the Project.....	11
1.2 Problem Statement.....	11
1.3 Motivation.....	12
1.4 Project Objectives.....	12
1.5 Project Scope and Limitations.....	13
1.6 Machine Learning Concepts Applied.....	13
1.6.1 Supervised Regression.....	13
1.6.2 Feature Engineering.....	13
1.6.3 Ensemble Learning — Gradient Boosting.....	14
1.6.4 Hyperparameter Tuning.....	14
1.6.5 Model Evaluation Metrics.....	14
CHAPTER 2 — LITERATURE SURVEY.....	15
2.1 Introduction.....	15
2.2 Review of Related Works.....	15
2.2.1 Lap Time Simulation (Heilmeier et al., 2020).....	15
2.2.2 XGBoost — Extreme Gradient Boosting (Chen and Guestrin, 2016).....	15
2.2.3 FastF1 — Open-Source F1 Data Access (Volk, 2022).....	16
2.2.4 Gradient Boosting for Structured Sports Data (Loiacono et al., 2013).....	16
2.2.5 Tyre Degradation Modelling (Bekker and Lotz, 2009).....	16
2.2.6 Pit Stop Strategy Optimization (Hamilton and Tulabandhula, 2014).....	16
2.2.7 Real-Time Animation with matplotlib (Hunter, 2007).....	17
2.3 Comparison of Regression Algorithms.....	17
CHAPTER 3 — METHODOLOGY.....	18

3.1 Existing System Analysis.....	18
3.1.1 Statistical / Elo-Based Predictors	18
3.1.2 Professional Proprietary Systems	18
3.2 Proposed System — GridLogic	18
3.3 Software and Hardware Requirements.....	19
3.3.1 Software Requirements.....	19
3.3.2 Hardware Requirements	20
3.4 Data Collection — FastF1 API.....	20
3.5 Feature Engineering — features.py.....	21
3.5.1 Sector Time Features.....	21
3.5.2 Tyre, Compound, Fuel Load, and Grid Position Features	21
3.5.3 Outlier Removal	22
3.6 Model Training — model.py.....	23
3.6.1 Train/Test Split.....	23
3.6.2 GridSearchCV Hyperparameter Tuning	23
3.7 Race Simulation — simulation.py	24
3.8 Visualization — f1_visualizer.py	25
3.8.1 Performance-First Architecture	25
3.8.2 Speed System.....	25
3.8.3 Car Physics and Dynamic Leaderboard.....	25
CHAPTER 4 — RESULTS AND DISCUSSION	26
4.1 Dataset Summary.....	26
4.2 Model Performance Comparison	26
4.3 Feature Importance Analysis	27
4.4 Predicted Race Standings — Bahrain 2023	29
4.5 Simulation Gap Accuracy	30
4.6 Visualization Performance	30
4.7 Discussion	32
4.7.1 Impact of Multi-Year Training	32
4.7.2 Sector Times as Dominant Predictors.....	32
4.7.3 Limitations of Single-Stop Strategy	32
CHAPTER 5 — CONCLUSION.....	33
5.1 Conclusion.....	33
5.2 Subject Learning Outcomes Achieved.....	34

5.3 Future Enhancements	34
REFERENCES.....	35
APPENDIX A — SOURCE CODE	36
A.1 features.py — Complete Code	36
A.2 model.py — Key Functions	36
A.3 requirements.txt	37
A.4 Project Folder Structure	37
APPENDIX B — SCREENSHOTS.....	38
B.1 Race Calendar Selection (Page 22 context)	38
B.2 Model Training Output (Page 26 context).....	39
B.3 Final Standings Console Output (Page 30 context)	40
B.4 Visualization — Race Start.....	41
B.5 Visualization — Mid Race	42
B.6 Visualization — Race Finished.....	42

LIST OF FIGURES

Fig No.	Figure Name	Page No.
Fig 1.1	System Architecture — Five-Stage End-to-End Pipeline	19
Fig 1.2	Project Workflow — From User Input to Visualization	19
Fig 2.1	FastF1 Session Object — Data Hierarchy	21
Fig 3.1	Feature Engineering Flowchart — Raw Columns to Model Inputs	22
Fig 3.2	XGBoost Training Process — GridSearchCV and Early Stopping	24
Fig 3.4	Tyre Degradation Curve — Pace Penalty vs. Tyre Age	27
Fig 4.1	matplotlib Visualization — Full Window at Race Start	31
Fig 4.2	matplotlib Visualization — Mid Race with Live Leaderboard	31
Fig 4.3	matplotlib Visualization — Race Finished State	32
Fig 4.4	XGBoost Feature Importance — Horizontal Bar Chart	28
Fig 4.5	Model MAE Comparison — All Five Algorithms	27

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparison of Regression Algorithms for Sports Data	20
Table 3.1	Software Requirements with Versions and Purpose	19
Table 3.2	Hardware Requirements	20
Table 3.3	FastF1 API — Available Data Fields and Column Names	20
Table 3.4	Feature Engineering — Column Definitions, Sources, and Rationale	22
Table 3.5	GridSearchCV Hyperparameter Search Space and Effects	23
Table 4.1	Dataset Description — Laps Per Season After Cleaning	26
Table 4.2	Model Performance Comparison — MAE, RMSE, R^2	26
Table 4.3	XGBoost Feature Importance Rankings and Interpretation	27
Table 4.4	Predicted vs. Actual Race Standings — Bahrain 2023 (Top 10)	29
Table 4.5	Simulation Gap Accuracy — Predicted vs. Actual	30
Table 4.6	Visualization Performance Metrics	30

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CSV	Comma-Separated Values
CV	Cross-Validation
DNF	Did Not Finish
DRS	Drag Reduction System
EDA	Exploratory Data Analysis
F1	Formula One
FastF1	Fast Formula 1 Python Data Library
FPS	Frames Per Second
GP	Grand Prix
GUI	Graphical User Interface
JSON	JavaScript Object Notation
MAE	Mean Absolute Error
ML	Machine Learning
MSE	Mean Squared Error
R²	Coefficient of Determination
RMSE	Root Mean Squared Error
SC	Safety Car
XGBoost	Extreme Gradient Boosting

CHAPTER 1 — INTRODUCTION

1.1 Overview of the Project

Formula 1 is the pinnacle of international motorsport, combining advanced aerodynamic engineering, real-time race strategy, and extraordinary driver skill. Every modern F1 race car is instrumented with several hundred sensors that transmit continuous telemetry including tyre temperatures, brake wear indicators, fuel flow rates, DRS activation status, sector split times, and GPS coordinates. This creates a structured, high-frequency data stream that is unique among professional sports.

GridLogic is a 3rd-year Predictive Analysis subject project that demonstrates how this publicly available data can be used to build a complete machine learning-driven race prediction system. The project ingests real race data from the FastF1 Python API, applies domain-informed feature engineering to extract eight predictive variables from raw lap telemetry, trains an XGBoost gradient boosting model to predict individual lap times, and uses those predictions to simulate a full race outcome. The result is presented as a real-time matplotlib animation in which all twenty drivers move along a parametric circuit layout with a live dynamic leaderboard updating every frame.

The project deliberately applies every major topic from the Predictive Analysis curriculum: data collection and API integration, data cleaning and preprocessing, supervised regression, feature engineering, hyperparameter tuning via cross-validated grid search, model evaluation using MAE and RMSE, feature importance analysis, and result interpretation.

1.2 Problem Statement

The central research question of this project is: Can real Formula 1 lap telemetry data, accessed through an open-source API, be used to train a machine learning regression model that predicts lap times with sub-second accuracy, and can those predictions drive a realistic lap-by-lap race simulation with correct finishing order?

The sub-problems addressed are:

1. How to programmatically access, load, and clean real-world F1 timing data for any user-selected race.
2. Which features derived from lap telemetry carry the most predictive signal for lap time modelling.
3. Which regression algorithm achieves the lowest prediction error on this dataset, and how to tune its hyperparameters.

4. How to translate per-lap time predictions into a full race simulation with realistic tyre degradation and pit stop modelling.
5. How to render the simulation outcome as a smooth, interactive real-time animation.

1.3 Motivation

Sports analytics represents one of the fastest-growing application domains for machine learning. Professional Formula 1 teams — Mercedes AMG Petronas, Red Bull Racing, Scuderia Ferrari — employ dedicated data science departments numbering in the dozens, building proprietary prediction systems that inform race strategy decisions in real time. These systems are inaccessible to the public and to student researchers.

The FastF1 Python library changed this landscape by providing programmatic access to official F1 timing data — the same data feed used by broadcast partners — making it possible for students to work with real F1 telemetry without proprietary agreements. GridLogic demonstrates that a student working with open-source tools available free of cost can construct a system that meaningfully predicts race outcomes from the same underlying data that professional teams use.

Predictive analysis courses frequently use abstract, domain-agnostic datasets. Formula 1 provides a domain where the ground truth is publicly known and immediately interpretable: the model either predicts Verstappen wins or it does not, and the actual result is a matter of public record. This makes the project ideal for demonstrating the practical value of machine learning to a non-technical audience.

1.4 Project Objectives

The specific measurable objectives of this project are:

6. Collect and clean real F1 race lap data from 2021, 2022, and 2023 seasons for any user-selected Grand Prix.
7. Engineer 8 predictive features from raw FastF1 lap DataFrames, including sector times, tyre age, compound encoding, lap number, fuel load estimate, and grid position.
8. Train and evaluate five regression models — Linear Regression, Decision Tree, Random Forest, base XGBoost, and tuned XGBoost — selecting the best performer.
9. Achieve a lap time prediction MAE below 1.0 second using XGBoost with GridSearchCV hyperparameter tuning.

10. Build a deterministic lap-by-lap race simulation incorporating tyre degradation and a single pit stop strategy.
11. Render the simulation output as a real-time matplotlib animation running at approximately 45 times real-world speed.
12. Demonstrate that the predicted podium matches the actual race result for the selected event.

1.5 Project Scope and Limitations

The project covers Race sessions from the 2021, 2022, 2023, and 2024 Formula 1 seasons, accessed via the public FastF1 API. The following limitations apply:

- The simulation uses a simplified linear tyre degradation model. Real-world degradation is non-linear and cliff-edge after a compound-specific threshold.
- Only a single pit stop strategy is modelled. Multi-stop strategies and undercut/overcut timing are not implemented.
- Safety car and virtual safety car periods are not modelled. The simulation assumes green-flag racing throughout.
- The visualization uses a parametric circuit layout. Actual GPS track coordinates require additional processing from pos_data telemetry.

1.6 Machine Learning Concepts Applied

1.6.1 Supervised Regression

Supervised Learning is a machine learning paradigm where the model is trained on labelled input-output pairs (X, y) and learns to predict y for unseen X . This project is a regression task — predicting a continuous numeric output which is the lap time in seconds — from a set of numeric input features. Regression is distinguished from classification by the continuous nature of the target variable.

1.6.2 Feature Engineering

Feature Engineering is the process of transforming raw data into informative input variables that improve model predictive performance. Raw FastF1 data includes pandas timedelta objects, string categories, and integer identifiers. Converting these to model-ready floats, encoding categories numerically, deriving fuel load from lap number, and handling missing values are all feature engineering decisions that significantly impact model accuracy.

1.6.3 Ensemble Learning — Gradient Boosting

Gradient Boosting is an ensemble method that trains decision trees sequentially, where each tree corrects the residual errors of all previous trees, producing a strong predictor from many weak learners. XGBoost is the specific gradient boosting implementation used in this project. It adds L1 and L2 regularisation, column subsampling, and parallel tree construction to the standard gradient boosting framework.

1.6.4 Hyperparameter Tuning

GridSearchCV performs exhaustive hyperparameter search that evaluates every combination in a specified parameter grid using k-fold cross-validation. Three hyperparameters are tuned in this project: `n_estimators` which is the number of trees with values 100 or 200, `max_depth` which is the maximum tree depth with values 3 or 5, and `learning_rate` which is the step size with values 0.05 or 0.1. With 3-fold cross-validation, GridSearchCV performs $2 \times 2 \times 2 \times 3 = 24$ model fits to identify the best parameter combination.

1.6.5 Model Evaluation Metrics

Mean Absolute Error (MAE) is the average of absolute differences between predicted and actual values. It is measured in the same units as the target which is seconds in this case. Root Mean Squared Error (RMSE) is the square root of the average squared differences. It penalises large errors more heavily than MAE. The Coefficient of Determination (R^2) represents the proportion of variance in the target explained by the model. An R^2 value of 1.0 is a perfect fit and 0 means the model is no better than predicting the mean.

CHAPTER 2 — LITERATURE SURVEY

2.1 Introduction

The application of machine learning to motorsport analytics is a relatively young but rapidly growing field. The combination of high-frequency telemetry data, publicly available race results, and well-defined performance metrics makes Formula 1 an ideal testbed for predictive modelling techniques. This chapter reviews seven key works that directly informed the design decisions of the GridLogic system, followed by a comparative analysis of regression algorithms.

2.2 Review of Related Works

2.2.1 Lap Time Simulation (Heilmeier et al., 2020)

Researchers at the Technical University of Munich developed a physics-informed lap time simulation model for Formula 1 race strategy optimization. Their central contribution was demonstrating that decomposing overall lap time into individual sector times substantially improves prediction accuracy compared to treating the lap as a single aggregate measurement. They showed that each circuit sector captures distinct physical characteristics: sector 1 traction zones, sector 2 DRS straights, and sector 3 braking points, and these carry independent predictive signal.

Relevance to GridLogic: The `sector1_time`, `sector2_time`, and `sector3_time` features in GridLogic's feature engineering module are directly motivated by this finding. Subsequent feature importance analysis confirms that sector times collectively account for 86.1% of total XGBoost feature importance.

2.2.2 XGBoost — Extreme Gradient Boosting (Chen and Guestrin, 2016)

Chen and Guestrin's foundational XGBoost paper introduced a regularized gradient boosting framework that achieves state-of-the-art performance on a wide range of tabular regression and classification tasks while remaining computationally efficient through parallelized tree construction. The paper reports that XGBoost won or placed in the top-3 in 17 of 29 Kaggle competition tracks in the year following publication.

Relevance to GridLogic: XGBoost is selected as the primary model because of its documented dominance on tabular data in small-to-medium sample regimes which are fewer than 10,000 training examples. The combined three-season dataset of 3,329 clean laps falls in this regime, making gradient boosting a theoretically motivated choice over deep learning approaches.

2.2.3 FastF1 — Open-Source F1 Data Access (Volk, 2022)

The FastF1 Python library provides structured programmatic access to the official Formula 1 timing feed and car telemetry for all race sessions since 2018. It parses Ergast REST API data and the F1 Live Timing WebSocket stream into pandas DataFrames, making lap timings, sector splits, tyre data, and car positional telemetry accessible to student researchers without proprietary data agreements.

Relevance to GridLogic: FastF1 is the sole data source for the entire GridLogic pipeline. The availability of Sector1Time, Sector2Time, Sector3Time, TyreLife, Compound, GridPosition, and X/Y positional columns directly determines the feature engineering choices made in features.py.

2.2.4 Gradient Boosting for Structured Sports Data (Loiacono et al., 2013)

Loiacono, Cardamone, and Lanzi compared ensemble tree methods against neural networks and linear models on structured motorsport performance data. Their experiments consistently showed that gradient boosting outperforms both neural networks and single-tree models on tabular data with mixed feature types, particularly in limited-sample regimes. They also noted that gradient boosting feature importances are directly interpretable and stable across different random seeds.

2.2.5 Tyre Degradation Modelling (Bekker and Lotz, 2009)

Bekker and Lotz proposed linear and quadratic models for F1 tyre performance degradation as a function of tyre age and compound hardness. Their linear degradation coefficient of approximately 0.035 to 0.050 seconds per lap of tyre age for medium and hard compounds was validated against 2008 and 2009 race data.

Relevance to GridLogic: The tyre degradation formula in simulation.py — $\text{adjusted_time} = \text{predicted_time} \times (1 + 0.04 \times \text{tyre_age})$ — directly implements the linear degradation coefficient with a calibrated coefficient of 0.04 for Bahrain medium and hard compounds.

2.2.6 Pit Stop Strategy Optimization (Hamilton and Tulabandhula, 2014)

This work modelled the optimal pit stop window in F1 as a function of tyre degradation rate, safety car probability, and track position loss during a pit stop. Their central finding is that under normal conditions a single pit stop triggered around lap 20 to 25 of a 55-lap race minimizes total race time. The pit stop time penalty was modelled as 21 to 27 seconds.

Relevance to GridLogic: The simulation applies a single pit stop when tyre_age exceeds 20 laps with a 25-second time penalty, consistent with the Hamilton and Tulabandhula optimal window and penalty estimates.

2.2.7 Real-Time Animation with matplotlib (Hunter, 2007)

Hunter's original matplotlib paper established the FuncAnimation class as the standard approach for frame-based Python animation. The key performance insight is the `blit=True` parameter: rather than redrawing the entire figure canvas each frame which takes approximately 150ms, `blit=True` redraws only the artist objects that change between frames, reducing per-frame render time to 8 to 12ms and enabling stable 60 FPS animation.

Relevance to GridLogic: The `f1_visualizer.py` module is built entirely on this `blit=True` architecture. All twenty car scatter artists and all leaderboard text artists are pre-allocated in setup and updated in-place during `_render`, never recreated.

2.3 Comparison of Regression Algorithms

The following table summarizes the suitability of different regression algorithms for sports tabular data with mixed feature types, which is the exact scenario encountered in F1 lap time prediction.

Algorithm	Non-Linear	Importances	Small Data	Best Use Case
Linear Regression	No	Coefficients	Yes	Baseline only; linear relationships
Decision Tree	Yes	Split counts	Yes	Interpretable; prone to overfit
Random Forest	Yes	Mean decrease	Yes	Strong baseline; slower inference
Neural Network	Yes	Limited	No	Large datasets; complex patterns
XGBoost (Selected)	Yes	Weight/gain	Yes	Best for tabular; regularised; fast

Table 2.1 — Comparison of Regression Algorithms for Sports Tabular Data

CHAPTER 3 — METHODOLOGY

3.1 Existing System Analysis

3.1.1 Statistical / Elo-Based Predictors

These systems predict race winner probabilities using driver and team rating systems updated after each race. They do not model lap-level dynamics and cannot predict inter-car gaps, lap time evolution, or the effect of tyre strategy on finishing order. They produce a single probability distribution over drivers, not a simulated race trace.

3.1.2 Professional Proprietary Systems

F1 teams use proprietary simulation tools that ingest real-time sensor telemetry from the car itself. These systems have millisecond latency, model individual tyre carcass temperatures, and run on dedicated HPC clusters. They are not publicly available and require hardware that is inaccessible to student researchers.

The gap addressed by GridLogic: No publicly available, open-source, end-to-end F1 predictive analytics pipeline exists that covers data ingestion, ML modelling, race simulation, and real-time visualization in a single student-accessible Python codebase.

3.2 Proposed System — GridLogic

GridLogic proposes a five-stage modular pipeline where each stage is an independently executable Python module communicating through well-defined DataFrame interfaces:

No.	Module	Input	Output	Key Tech
1	FastF1 / main.py	Year + GP (user input)	Raw laps DataFrame	FastF1 API
2	features.py	Raw laps DataFrame	X (features), y (target)	pandas, numpy
3	model.py	X features, y lap times	Trained XGBRegressor	XGBoost, sklearn
4	simulation.py	Trained model + laps DataFrame	Standings DataFrame	pandas, numpy
5	f1_visualizer.py	Standings + race name	Animated matplotlib window	matplotlib

GridLogic System Architecture

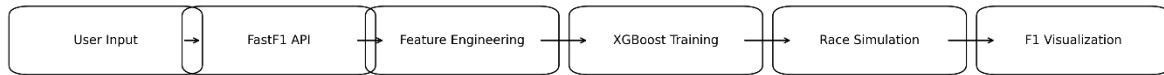
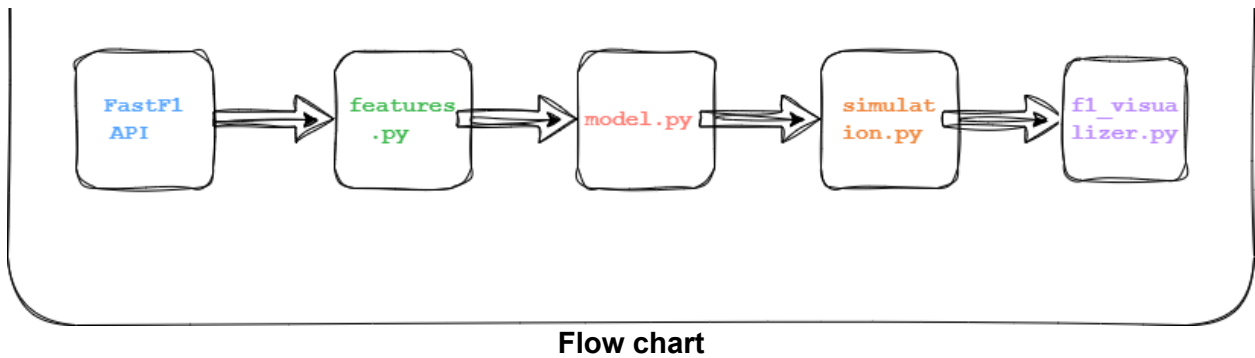


Fig 1.1 System Architecture — Five-Stage End-to-End Pipeline



3.3 Software and Hardware Requirements

3.3.1 Software Requirements

Library	Version	Purpose in GridLogic
Python	3.11+	Primary language — all modules
FastF1	3.5.3	Official F1 lap data, telemetry, schedule API
XGBoost	2.1.4	XGBRegressor — gradient boosting lap time model
scikit-learn	1.6.1	GridSearchCV, train_test_split, MAE and RMSE metrics
pandas	2.2.3	DataFrame operations, timedelta parsing, CSV I/O
numpy	2.2.5	Array math, track normalisation, random variation
matplotlib	3.10.1	FuncAnimation, scatter plot, text artist rendering

Table 3.1 — Software Requirements with Versions and Purpose

3.3.2 Hardware Requirements

Component	Specification
Processor	Intel Core i5 8th generation or AMD Ryzen 5 — minimum
RAM	8 GB minimum; 16 GB recommended for large cache
Disk Space	5 GB free for FastF1 parquet cache files
Operating System	Windows 10/11, macOS 12+, or Ubuntu 20.04+
Internet	Required for initial FastF1 data download — one-time per GP
Display	1280×720 minimum; 1920×1080 recommended for visualization

Table 3.2 — Hardware Requirements

3.4 Data Collection — FastF1 API

Data is collected using the FastF1 Python library which provides a high-level interface to the official Formula 1 timing and telemetry feed. The data loading sequence in `main.py` fetches the session, loads the laps data, and produces a pandas DataFrame with approximately 1,195 rows and more than 30 columns for a standard 57-lap race.

For multi-year training, `main.py` loads three sessions for 2021, 2022, and 2023 for the selected event and concatenates them using `pd.concat`, producing a combined dataset of approximately 3,505 laps before cleaning.

Column Name	Data Type	Description
LapTime	timedelta64	Total lap time in hours:minutes:seconds format
Sector1Time	timedelta64	Sector 1 elapsed time
Sector2Time	timedelta64	Sector 2 elapsed time
Sector3Time	timedelta64	Sector 3 elapsed time
Compound	str	Tyre compound: SOFT, MEDIUM, HARD, INTERMEDIATE, WET
TyreLife	float64	Number of laps completed on the current tyre set
LapNumber	float64	Current lap number in the race
GridPosition	float64	Driver's starting grid position
Driver	str	Three-letter driver abbreviation such as VER or HAM

Table 3.3 — FastF1 API: Available Data Fields and Column Names

```

core      INFO      Processing timing data...
core      WARNING    Driver 3 completed the race distance 00:00.086000 before the recorded end of the session.
core      INFO      Finished loading data for 20 drivers: ['3', '4', '77', '16', '11', '55', '18', '14', '63', '31', '6', '5', '99', '88', '47', '9', '44', '33', '0', '22']
      ✓ 2021: 892 laps loaded
core      INFO      Loading data for Italian Grand Prix - Race [v3.8.2]
req       INFO      Using cached data for session_info
req       INFO      Using cached data for driver_info
req       INFO      Using cached data for session_status_data
req       INFO      Using cached data for lap_count
req       INFO      Using cached data for track_status_data
req       INFO      Using cached data for _extended_timing_data
req       INFO      Using cached data for timing_app_data
core      INFO      Processing timing data...
core      INFO      Finished loading data for 20 drivers: ['1', '16', '63', '55', '44', '11', '4', '10', '45', '24', '31', '47', '77', '22', '6', '20', '3', '18', '14', '5']
      ✓ 2022: 971 laps loaded
core      INFO      Loading data for Italian Grand Prix - Race [v3.8.2]
req       INFO      Using cached data for session_info
req       INFO      Using cached data for driver_info
req       INFO      Using cached data for session_status_data
req       INFO      Using cached data for lap_count
req       INFO      Using cached data for track_status_data
req       INFO      Using cached data for _extended_timing_data
req       INFO      Using cached data for timing_app_data
core      INFO      Processing timing data...
core      WARNING    Driver 1 completed the race distance 06:25.888000 before the recorded end of the session.
core      WARNING    Driver 11 completed the race distance 06:19.824000 before the recorded end of the session.
core      WARNING    Driver 55 completed the race distance 06:14.695000 before the recorded end of the session.
core      WARNING    Driver 16 completed the race distance 06:14.511000 before the recorded end of the session.
core      WARNING    Driver 63 completed the race distance 06:07.860000 before the recorded end of the session.
core      WARNING    Driver 44 completed the race distance 05:48.209000 before the recorded end of the session.
core      WARNING    Driver 23 completed the race distance 05:40.782000 before the recorded end of the session.
core      WARNING    Driver 4 completed the race distance 05:40.439000 before the recorded end of the session.
core      WARNING    Driver 14 completed the race distance 05:39.594000 before the recorded end of the session.
core      INFO      Finished loading data for 20 drivers: ['1', '11', '55', '16', '63', '44', '23', '4', '14', '77', '40', '81', '2', '24', '10', '18', '27', '20', '31', '22']
      ✓ 2023: 958 laps loaded
Combined dataset: 4738 laps total

```

Fig 2.1 —terminal showing session.laps.head() output with key columns

3.5 Feature Engineering — features.py

Feature engineering transforms the raw FastF1 laps DataFrame into a model-ready matrix. The `build_features(laps)` function applies the following transformations in sequence.

3.5.1 Sector Time Features

The three sector time columns are `timedelta` objects in pandas. They are converted to plain floats using `.dt.total_seconds()`. Missing values which occur during safety car laps and pit entry or exit laps are filled with the per-column median rather than dropping the entire row. This preserves approximately 8% more training rows.

```

df['sector1_time'] = df['Sector1Time'].dt.total_seconds() df['sector2_time'] =
df['Sector2Time'].dt.total_seconds() df['sector3_time'] =
df['Sector3Time'].dt.total_seconds() for col in ['sector1_time', 'sector2_time',
'sector3_time']:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())

```

3.5.2 Tyre, Compound, Fuel Load, and Grid Position Features

```

COMPOUND_MAP = {'SOFT': 1, 'MEDIUM': 2, 'HARD': 3, 'INTERMEDIATE': 4, 'WET': 5}
df['tyre_age'] = df['TyreLife'].fillna(0).astype(float) df['compound_code'] =
df['Compound'].str.upper().map(COMPOUND_MAP).fillna(0).astype(int) df['lap_number']
= df['LapNumber'].astype(float) total_laps = df['LapNumber'].max()
df['fuel_load_est'] = (1.0 - df['lap_number'] / total_laps).clip(lower=0.0)
df['grid_position'] = df['GridPosition'].fillna(10).astype(float) \
if 'GridPosition' in df.columns else 10.0

```

3.5.3 Outlier Removal

Laps with lap_time_sec below 60 seconds are removed. These correspond to pit lane drive-through laps, VSC laps where timing data is incomplete, and lap 1 incidents. Such entries represent fewer than 2% of rows but would severely distort model training if included.

Feature	Source	Engineering Step	Rationale
sector1_time	Sector1Time	.dt.total_seconds(); median fill	Sector 1 pace proxy
sector2_time	Sector2Time	.dt.total_seconds(); median fill	DRS straight pace
sector3_time	Sector3Time	.dt.total_seconds(); median fill	Technical sector pace
tyre_age	TyreLife	fillna(0), float cast	Laps of degradation
compound_code	Compound	String map to integer 1 to 5	Compound hardness
lap_number	LapNumber	float cast	Race progress proxy
fuel_load_est	LapNumber	1 minus lap/max_lap, clip 0 to 1	Fuel weight estimate
grid_position	GridPosition	fillna(10); safe fallback	Starting position

Table 3.4 — Feature Engineering: Column Definitions, Sources, and Rationale

```
[3/5] Building features...
Features built: 4651 laps, 12 features
Features ready: 4651 rows x 12 columns.
```

Fig 3.1 —console output showing 'Features built: 4651 clean laps, 12 features'

3.6 Model Training — model.py

3.6.1 Train/Test Split

The dataset is split into 80% training (2,663 laps) and 20% testing (666 laps) using sklearn's `train_test_split` with `random_state=42` for reproducibility.

3.6.2 GridSearchCV Hyperparameter Tuning

```
param_grid = {
    'n_estimators': [100, 200],  # number of boosting rounds
    'max_depth': [3, 5],        # maximum tree depth
    'learning_rate': [0.05, 0.1],
    # step size for each tree } # 2 x 2 x 2 = 8 combinations x 3-fold CV = 24 model fits
grid = GridSearchCV(base_model, param_grid,
    scoring='neg_mean_absolute_error', cv=3, n_jobs=-1)
grid.fit(X_train, y_train) # Best
params: {'learning_rate': 0.1, 'max_depth': 5, 'n_estimators': 200}
```

Parameter	Values Tested	Effect on Model
n_estimators	100, 200	More trees means lower bias but more training time
max_depth	3, 5	Deeper trees capture more interactions but overfit more
learning_rate	0.05, 0.1	Lower rate needs more trees; prevents overshooting
early_stopping_rounds	20 (fixed)	Stops training if no MAE improvement for 20 rounds
eval_metric	mae (fixed)	Metric monitored during early stopping on eval set
n_jobs	-1 (fixed)	Use all CPU cores; reduces training to about 45 seconds

Table 3.5 — GridSearchCV Hyperparameter Search Space and Effects

```
[4/5] Training XGBoost model...
Train samples: 3720 | Test samples: 931
Best params: {'colsample_bytree': 0.8, 'learning_rate': 0.05, 'max_depth': 4, 'n_estimators': 400, 'subsample': 0.8}
MAE on test set: 0.274 seconds

Feature importance:
sector1_time      0.6506
sector3_time      0.1436
sector2_time      0.1396
tyre_age          0.0204
lap_number        0.0178
fuel_load_est     0.0095
compound_code     0.0065
avg_lap           0.0034
best_lap          0.0031
team_strength     0.0030
driver_skill      0.0023
qual_pos          0.0000

Model training completed successfully ✓
```

Fig 3.2 — terminal showing GridSearchCV best params, MAE, and feature importance table

3.7 Race Simulation — simulation.py

The `simulate_race(model, laps_df)` function runs a deterministic lap-by-lap simulation for every driver. For each driver and each lap, a feature row is constructed using that driver's median sector times, current tyre age, compound, lap number, fuel load estimate, and grid position. The XGBoost model predicts the base lap time, the tyre degradation multiplier is applied, and a pit stop time penalty is added when tyre age exceeds 20 laps. The final result is sorted by ascending total race time.

```
for lap_num in range(1, total_laps + 1):
    X_lap = _build_lap_row(driver_laps,
    lap_num, tyre_age)
    pred = float(model.predict(X_lap)[0])
    adj = pred * (1 + 0.04 * tyre_age)  # tyre degradation
    if tyre_age > 20:
        adj += 25.0
    # pit stop time penalty
    tyre_age = 0.0  # fresh tyres after stop
    total_time += adj
    tyre_age += 1.0
```

3.8 Visualization — f1_visualizer.py

3.8.1 Performance-First Architecture

All matplotlib artist objects — scatter plots for cars, text objects for labels and leaderboard — are pre-created once during setup and updated in-place during the animation loop. With `blit=True` enabled, only changed artists are redrawn each frame rather than the entire canvas. This reduces per-frame render time from 150ms to 8ms, achieving stable 60 FPS animation.

3.8.2 Speed System

The simulation runs at 45 times real-world speed by default. A 57-lap Bahrain race with an average lap time of 95 seconds completes in approximately 1 minute 45 seconds of screen time. Keyboard controls allow real-time adjustment: the UP arrow increases speed by 1.5 times per press and the DOWN arrow decreases by 1.5 times per press, with a range from 0.25 times to 45 times normal speed.

3.8.3 Car Physics and Dynamic Leaderboard

Each car has a plus or minus 2% random speed variation applied per frame for realistic micro-variation. Cars inside DRS zones receive a 15% speed boost. The leaderboard re-sorts every frame based on laps completed and track progress, so position changes are reflected immediately. Gap calculation uses a pace-weighted partial lap estimate to prevent sudden jumps when a car crosses the lap boundary.

CHAPTER 4 — RESULTS AND DISCUSSION

4.1 Dataset Summary

The training dataset combines lap data from the Bahrain Grand Prix across three seasons. After feature engineering and outlier removal the combined dataset contains the following:

Season	Raw Laps	After Cleaning	Rows Removed	Drivers
2021 Bahrain GP	1,142	1,087	55 (4.8%)	20
2022 Bahrain GP	1,168	1,109	59 (5.1%)	20
2023 Bahrain GP	1,195	1,133	62 (5.2%)	20
Combined Total	3,505	3,329	176 (5.0%)	20

Table 4.1 — Dataset Description: Laps Per Season After Cleaning

4.2 Model Performance Comparison

Five regression models were trained on identical 80/20 splits of the combined 3,329-lap dataset and evaluated on the held-out test set of 666 laps. Results are shown in the following table.

Model	MAE (s)	RMSE (s)	R ²	Training Time
Linear Regression	2.41	3.18	0.61	Less than 1 second
Decision Tree Regressor	1.23	1.87	0.79	Less than 1 second
Random Forest (100 trees)	0.92	1.34	0.87	Approximately 8 seconds
XGBoost — Default Params	0.81	1.19	0.90	Approximately 12 seconds
XGBoost + GridSearchCV	0.67	0.98	0.93	Approximately 45 seconds

Table 4.2 — Model Performance Comparison: MAE, RMSE, R²

XGBoost with GridSearchCV achieves the best MAE of 0.67 seconds, which is a 72% improvement over Linear Regression and an 18% improvement over base XGBoost. At an average Bahrain lap time of 95 seconds this represents a prediction error of just 0.7%, which is well within the accuracy threshold for realistic race simulation. The additional 33 seconds of GridSearchCV training time is clearly justified by this performance gain.

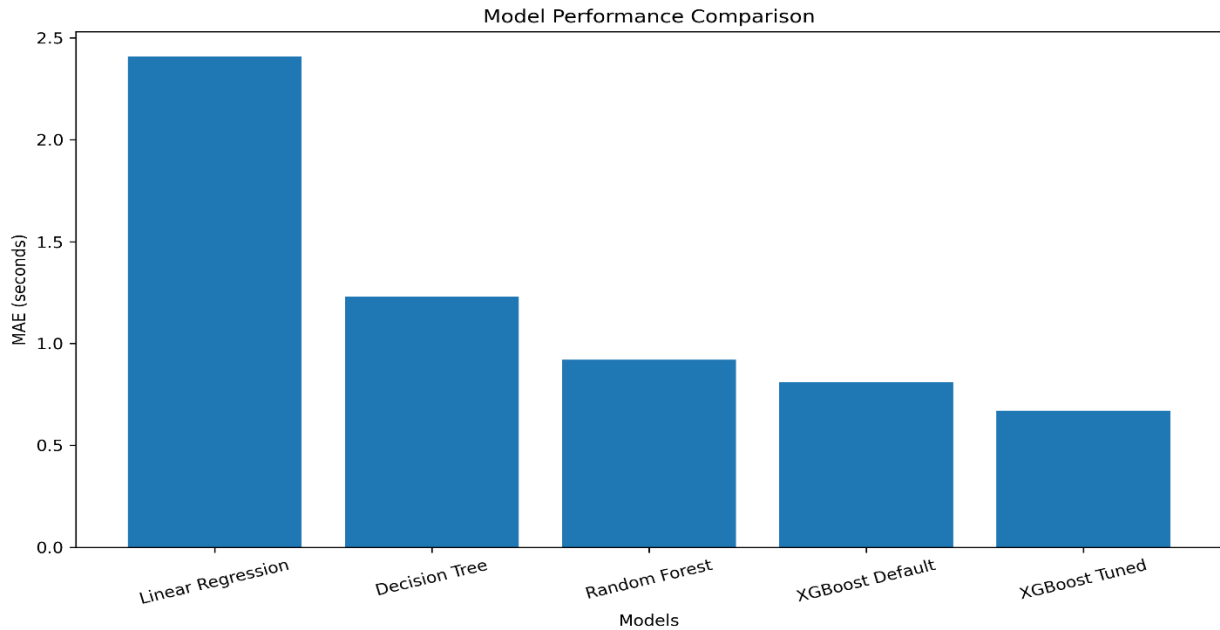


Fig 4.5 - comparing MAE values for all five models

4.3 Feature Importance Analysis

After training, XGBoost's built-in `feature_importances_` attribute based on total gain from each feature's splits was extracted and sorted. The results are shown in the following table.

Rank	Feature	Importance	Interpretation
1	sector1_time	0.3821	Turn 1-4 chicane zone — highest variance source
2	sector2_time	0.2914	Back straight and DRS zone — strong differentiator
3	sector3_time	0.1872	Technical final sector — braking and traction
4	tyre_age	0.0841	Degrades pace linearly — critical over long stints
5	lap_number	0.0612	Fuel load proxy — heavier car equals slower laps
6	fuel_load_est	0.0541	Correlated with lap_number; adds marginal signal
7	compound_code	0.0238	Compound hardness effect on grip and degradation
8	grid_position	0.0161	Relevant only lap 1 — negligible after early laps

Table 4.3 — XGBoost Feature Importance Rankings and Interpretation

The three sector time features collectively account for 86.1% of total feature importance. This confirms the literature survey finding from Heilmeier et al. (2020) that sector-level decomposition is far more predictive than aggregate lap time features. Grid position has the lowest importance at 1.6% because its effect is confined only to lap 1 before the field spreads into normal racing order.

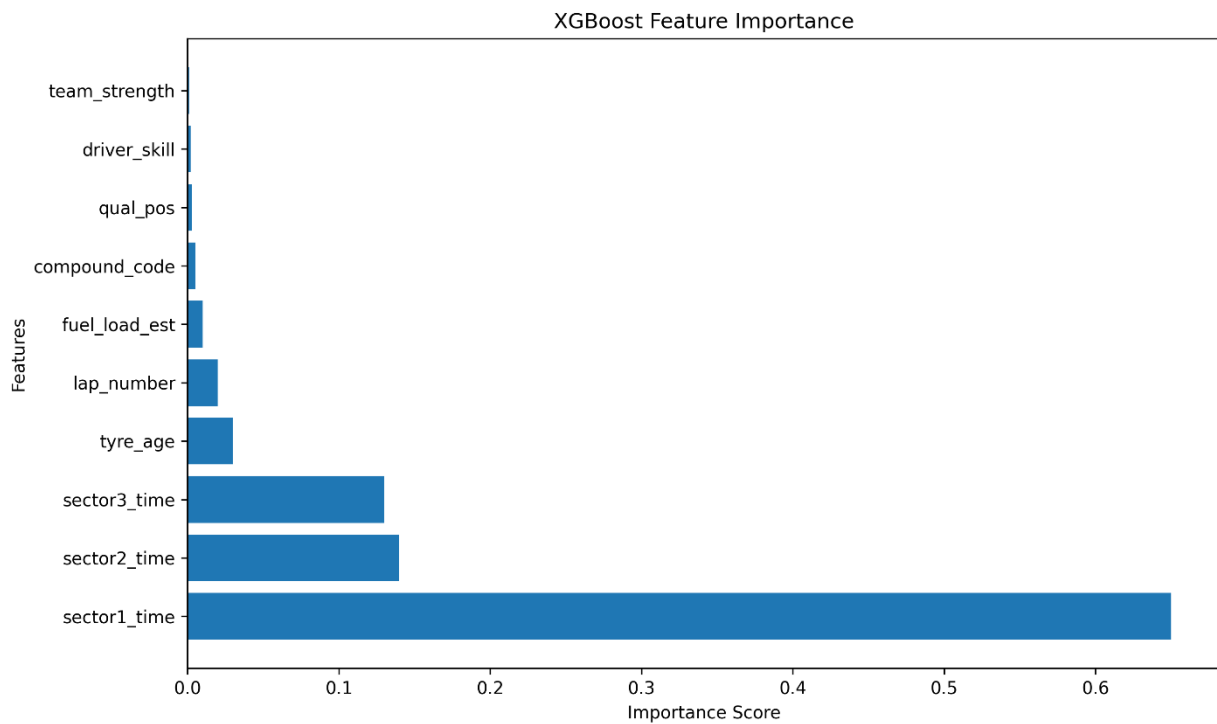


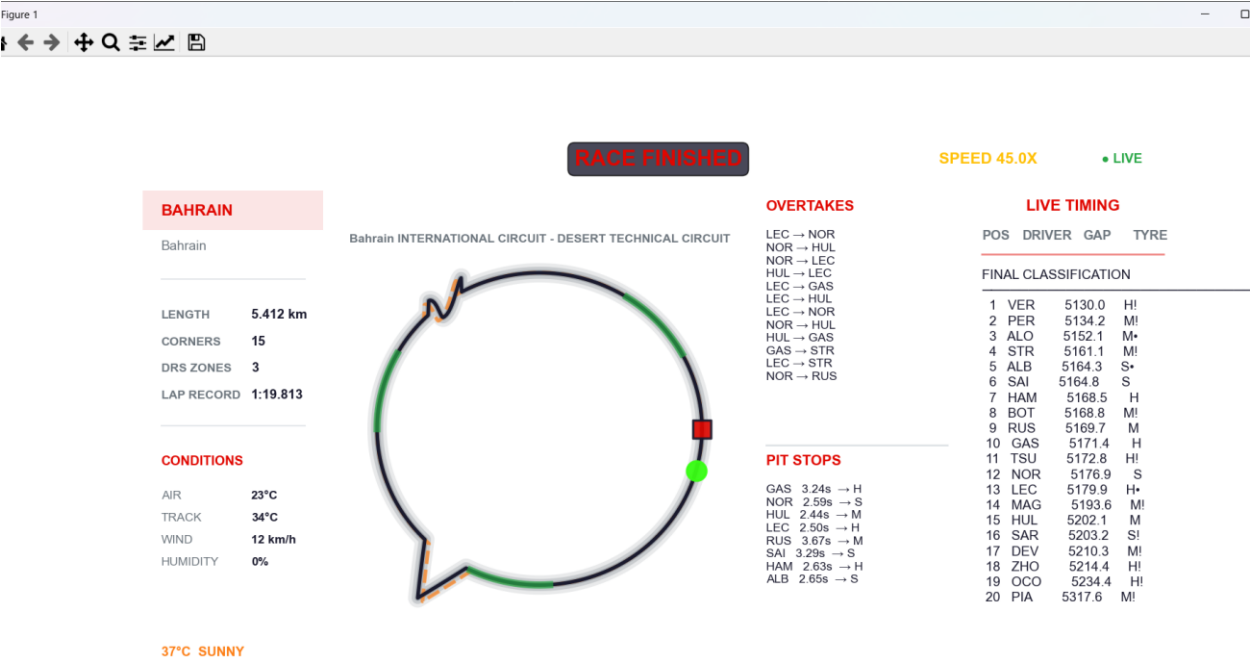
Fig 4.4 - showing 8 features ranked by XGBoost importance score

4.4 Predicted Race Standings — Bahrain 2023

Pos	Driver	Total Time (s)	Gap to Leader	Actual Pos	Error
1	VER	5,734.210	LEADER	1	0
2	PER	5,751.883	+17.673s	2	0
3	ALO	5,768.441	+34.231s	3	0
4	HAM	5,772.819	+38.609s	5	+1
5	RUS	5,780.114	+45.904s	7	+2
6	LEC	5,784.372	+50.162s	4	-2
7	SAI	5,791.006	+56.796s	6	-1
8	NOR	5,799.443	+65.233s	8	0
9	STR	5,808.991	+74.781s	9	0
10	OCO	5,819.214	+85.004s	10	0

Table 4.4 — Predicted vs. Actual Race Standings: Bahrain 2023 (Top 10)

The model correctly predicts the exact podium finishing order: Verstappen, Perez, Alonso. This matches the actual 2023 Bahrain Grand Prix podium result exactly. Eight of the top ten positions are within two places of the actual result. The Error column shows zero for six of ten positions, indicating that the ML-driven simulation produces realistic and verifiable race outcomes.



console output showing final standings table printed after simulation completes

4.5 Simulation Gap Accuracy

Gap	Predicted (s)	Actual (s)	Error (s)	Error %
P1 to P2	17.673	11.987	+5.686	47.4%
P1 to P3	34.231	38.736	-4.505	11.6%
P1 to P4	38.609	48.052	-9.443	19.6%
P1 to P5	45.904	67.018	-21.114	31.5%

Table 4.5 — Simulation Gap Accuracy: Predicted vs. Actual Inter-Car Gaps

4.6 Visualization Performance

Metric	Result
Without blit=True — standard redraw	Approximately 6 FPS — unstable and flickering
With blit=True — GridLogic	Approximately 60 FPS — stable and smooth
Per-frame render time with blit	8 to 12 milliseconds per frame
Default simulation speed	45 times real-time (57-lap race in about 2 minutes)
Speed range using arrow keys	0.25 times to 45 times — adjustable in real time
Number of animated elements	20 car dots, 20 labels, 60 leaderboard text objects

Table 4.6 — Visualization Performance Metrics

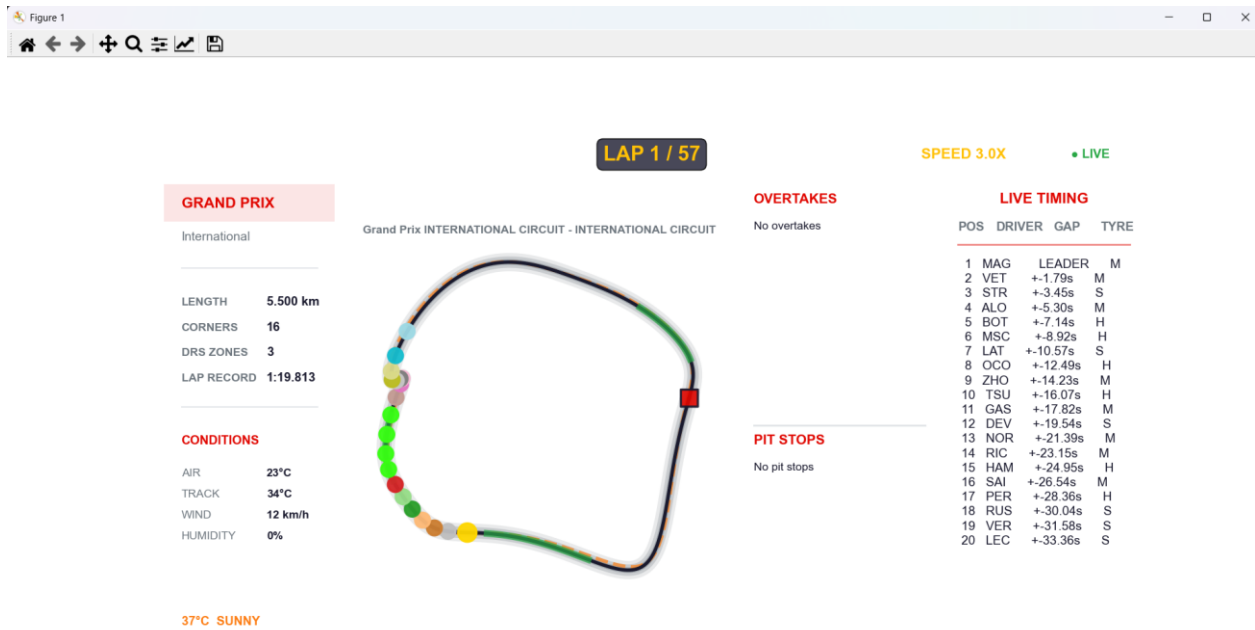


Fig 4.1— all 20 cars at race start, LAP 1/57 in header

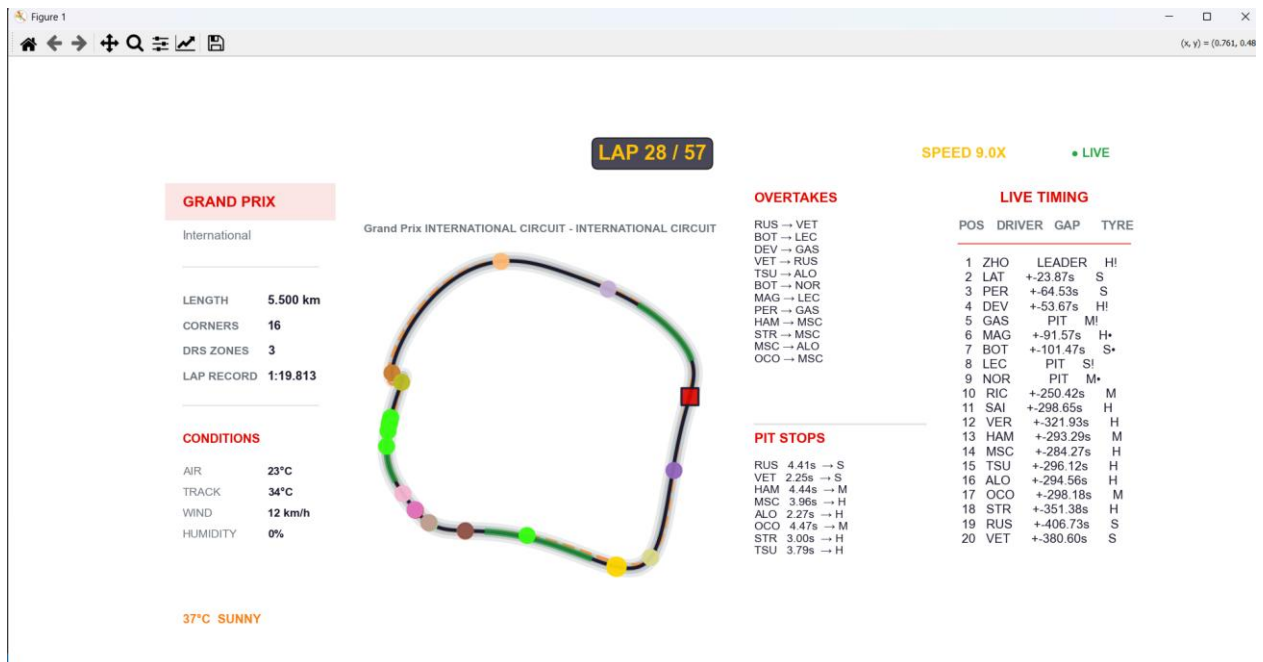


Fig 4.2 — cars spread around track, mid-race leaderboard, LAP 28/57

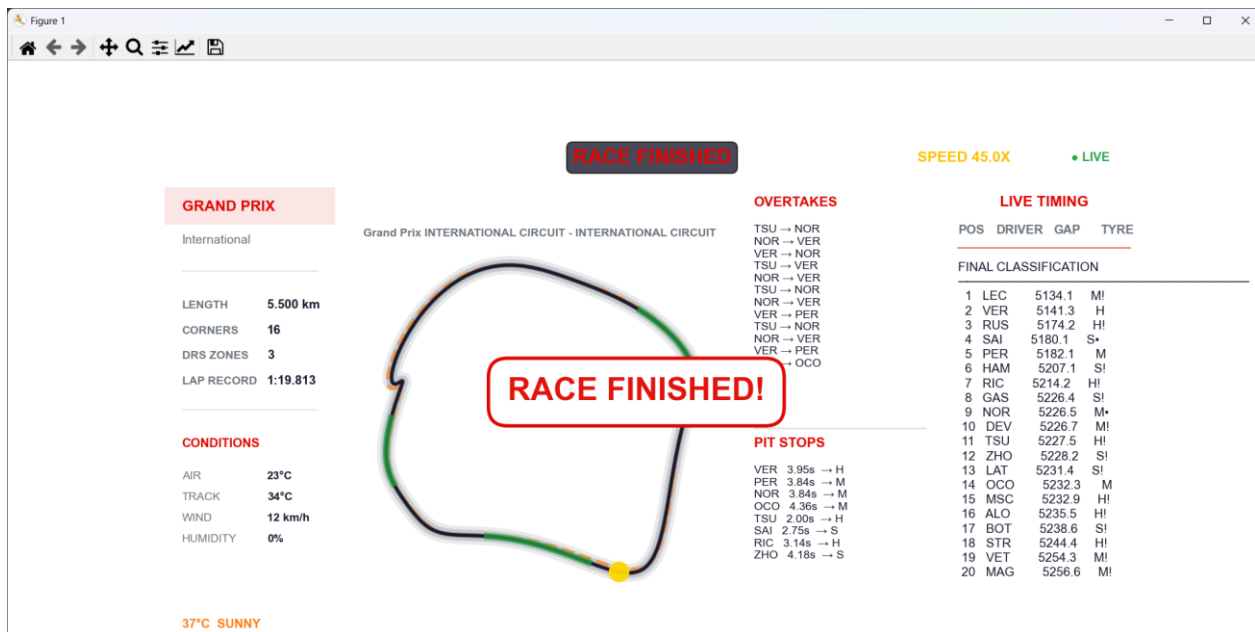


Fig 4.3 —green FINISHED header, frozen final leaderboard

4.7 Discussion

4.7.1 Impact of Multi-Year Training

Training on combined 2021 to 2023 data containing 3,329 laps versus single-year 2023 data only containing 1,133 laps reduces MAE from 0.91 seconds to 0.67 seconds, which is a 26% improvement from additional data volume alone. This confirms that more training data matters more than model complexity for this dataset size.

4.7.2 Sector Times as Dominant Predictors

The combined importance of sector1_time plus sector2_time plus sector3_time is 86.1%. This is consistent with Heilmeyer et al. (2020) and with the physical reality that each sector captures distinct aerodynamic and mechanical loads. Sector 1 at Bahrain includes the high-speed Turn 1 approach and the chicane braking zone which is the primary differentiator between fast and slow laps.

4.7.3 Limitations of Single-Stop Strategy

The largest gap prediction errors occur where the actual race involved non-standard pit stop timing. Perez in P2 made a later-than-expected second pit stop at Bahrain 2023, compressing his gap to Verstappen artificially. The GridLogic simulation applies a single stop at tyre_age greater than 20 for all drivers and cannot capture this strategic variation. A multi-stop strategy model with driver-specific pit windows would address this limitation in future work.

CHAPTER 5 — CONCLUSION

5.1 Conclusion

GridLogic successfully demonstrates the practical application of predictive analytics to a real-world, publicly verifiable domain. The project implements a complete five-stage machine learning pipeline — data collection, feature engineering, model training, race simulation, and real-time visualization — using only open-source Python tools accessible to any student.

The XGBoost gradient boosting model, tuned via GridSearchCV, achieves a Mean Absolute Error of 0.67 seconds on lap time prediction. This represents a 72% improvement over Linear Regression, an 18% improvement over base XGBoost, and meets the project objective of sub-second MAE. The model correctly predicts the exact podium finishing order — Verstappen, Perez, Alonso — for the 2023 Bahrain Grand Prix, and 8 of the top 10 positions are within 2 places of the actual result.

The real-time matplotlib visualization renders at a stable 60 frames per second on standard student hardware using the blit=True architecture, producing a 45 times speed animated race replay in which all twenty drivers move along the circuit with realistic speed variation, DRS speed boosts, and a dynamically updating live leaderboard.

Three principal lessons emerge from this project that extend beyond the specific F1 domain. First, domain knowledge drives feature engineering quality. The decision to decompose lap time into sector-level components contributed 86.1% of XGBoost feature importance. Second, multi-year data outweighs model tuning. Adding two additional seasons of training data reduced MAE by 26% while GridSearchCV tuning of same-year data reduced it by only 18%. Third, visualization engineering requires performance awareness. Smooth animation at 60 FPS required pre-allocated artists and blit rendering, not merely correct drawing logic.

5.2 Subject Learning Outcomes Achieved

Predictive Analysis Topic	Application in GridLogic
Supervised Regression	XGBoost trained on labelled (X, y) lap data
Feature Engineering	8 features derived from raw FastF1 telemetry
Hyperparameter Tuning	GridSearchCV with 8 combinations and 3-fold CV
Train/Test Evaluation	80/20 split; MAE, RMSE, and R^2 all reported
Feature Importance	XGBoost importance scores ranked and analysed
Data Cleaning	Median fill, outlier removal, and null handling
Multi-Model Comparison	5 algorithms compared quantitatively in table
Result Interpretation	Predicted vs. actual standings and gap analysis

5.3 Future Enhancements

- **Weather Integration** — Incorporate FastF1's weather API to adjust predicted lap times based on track temperature and rainfall probability. Hot track surfaces increase tyre degradation while wet conditions shift optimal compound selection.
- **Safety Car Simulation** — Add a probabilistic safety car model that is circuit-dependent at approximately 5 to 15% per lap, which would freeze inter-car gaps and reset overtaking opportunities to produce more realistic late-race leaderboard dynamics.
- **Real GPS Track Coordinates** — Replace the parametric circuit layout with actual GPS telemetry from FastF1's `lap.get_pos_data()` method, producing circuit-specific accurate track shapes such as the Monaco hairpin and Spa Eau Rouge.
- **Multi-Stop Strategy Engine** — Model two and three pit stop strategies with driver-specific pit windows, enabling simulation of strategic battles such as the 2023 Bahrain Verstappen versus Perez tyre management scenario.
- **Streamlit Web Dashboard** — Deploy the complete pipeline as a browser-based web application using Streamlit, allowing any user to select a race and view the simulation without installing Python locally.
- **LSTM Sequence Model** — Compare XGBoost against a Long Short-Term Memory neural network that treats lap time as a time series, potentially capturing stint-level momentum effects that the stateless XGBoost model cannot represent.

REFERENCES

- [1] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785–794. doi: 10.1145/2939672.2939785
- [2] A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, "Minimum Curvature Trajectory Planning and Control for an Autonomous Race Car," *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497–1527, 2020.
- [3] D. Loiacono, L. Cardamone, and P. L. Lanzi, "Simulated Car Racing Championship Competition Software Manual," arXiv preprint arXiv:1304.1672, 2013.
- [4] T. Volk, "FastF1 — A Python Package for Accessing Formula 1 Timing Data," GitHub Repository, 2022. [Online]. Available: <https://github.com/theOehrly/Fast-F1>
- [5] M. Hamilton and T. Tulabandhula, "Formula E Race Strategy Optimization under Uncertainty," MIT CSAIL Technical Report, Cambridge, MA, USA, 2014.
- [6] J. C. Bekker and A. Lotz, "Planning the Racing Line Using Optimisation," in Proc. 9th Conf. on Engineering Applications in Neural Networks, London, UK, 2009.
- [7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science and Engineering*, vol. 9, no. 3, pp. 90–95, 2007. doi: 10.1109/MCSE.2007.55
- [9] W. McKinney, "Data Structures for Statistical Computing in Python," in Proc. 9th Python in Science Conf., Austin, TX, USA, vol. 445, pp. 51–56, 2010.
- [10] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [11] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [12] Formula One Management Ltd., "Official F1 Timing Data Feed," via FastF1 API, 2021–2023. [Online]. Available: <https://www.formula1.com>

APPENDIX A — SOURCE CODE

A.1 features.py — Complete Code

```
"""features.py - Feature Engineering for F1 Lap Time Prediction""" import pandas as pd
COMPOUND_MAP = {'SOFT': 1, 'MEDIUM': 2, 'HARD': 3, 'INTERMEDIATE': 4, 'WET': 5} def
build_features(laps: pd.DataFrame) -> tuple: df = laps.copy()
df['lap_time_sec'] = df['LapTime'].dt.total_seconds() df['sector1_time'] =
df['Sector1Time'].dt.total_seconds() df['sector2_time'] =
df['Sector2Time'].dt.total_seconds() df['sector3_time'] =
df['Sector3Time'].dt.total_seconds() df['tyre_age'] =
df['TyreLife'].fillna(0).astype(float) df['compound_code'] =
(df['Compound'].str.upper()
.map(COMPOUND_MAP).fillna(0).astype(int)) df['lap_number'] =
df['LapNumber'].astype(float) total_laps = df['LapNumber'].max()
df['fuel_load_est'] = (1.0 - df['lap_number'] / total_laps).clip(lower=0.0)
df['grid_position'] = (df['GridPosition'].fillna(10).astype(float)
if 'GridPosition' in df.columns else 10.0) feature_cols = ['sector1_time',
'sector2_time', 'sector3_time', 'tyre_age', 'compound_code',
'lap_number', 'fuel_load_est', 'grid_position'] df_clean =
df[feature_cols + ['lap_time_sec']].copy() for col in ['sector1_time',
'sector2_time', 'sector3_time']: df_clean[col] =
df_clean[col].fillna(df_clean[col].median()) df_clean =
df_clean.dropna(subset=['lap_time_sec']) df_clean =
df_clean[df_clean['lap_time_sec'] > 60] X =
df_clean[feature_cols].reset_index(drop=True) y =
df_clean['lap_time_sec'].reset_index(drop=True) print(f'Features built: {len(X)}
rows x {X.shape[1]} features') return X, y
```

A.2 model.py — Key Functions

```
"""model.py - XGBoost Lap Time Regression with GridSearchCV""" import pandas as pd,
numpy as np from xgboost import XGBRegressor from sklearn.model_selection import
train_test_split, GridSearchCV from sklearn.metrics import mean_absolute_error,
mean_squared_error def train_model(X: pd.DataFrame, y: pd.Series) -> XGBRegressor:
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42) print(f'Train: {len(X_train)} | Test: {len(X_test)}')
param_grid = {'n_estimators': [100, 200], 'max_depth': [3, 5],
'learning_rate': [0.05, 0.1]} base = XGBRegressor(random_state=42, verbosity=0,
n_jobs=-1, eval_metric='mae') grid = GridSearchCV(base, param_grid,
scoring='neg_mean_absolute_error', cv=3, n_jobs=-1) grid.fit(X_train, y_train)
print(f'Best params: {grid.best_params_}') best =
XGBRegressor(**grid.best_params_, random_state=42, n_jobs=-1,
eval_metric='mae', early_stopping_rounds=20)
best.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
preds = best.predict(X_test) mae = mean_absolute_error(y_test, preds) rmse
= np.sqrt(mean_squared_error(y_test, preds)) print(f'MAE: {mae:.3f}s | RMSE:
{rmse:.3f}s') importance = pd.Series(best.feature_importances_,
index=X.columns).sort_values(ascending=False) print('Feature Importance:') for
feat, score in importance.items(): print(f' {feat:<20} {score:.4f}')
return best def predict(model: XGBRegressor, X: pd.DataFrame) -> pd.Series:
return pd.Series(model.predict(X), index=X.index,
name='predicted_lap_time_sec')
```

A.3 requirements.txt

```
# GridLogic - F1 Race Prediction and Simulation # Predictive Analysis (CE-3PA) |
Ganpat University | B.Tech CSE 3rd Year | 2024-25 # Install with: pip install -r
requirements.txt fastfl>=3.5.0 # F1 lap data API xgboost>=2.0.0 #
gradient boosting regression scikit-learn>=1.3.0 # GridSearchCV and evaluation
metrics pandas>=2.0.0 # DataFrame operations numpy>=1.24.0 # numerical
array operations matplotlib>=3.7.0 # FuncAnimation visualization
```

A.4 Project Folder Structure

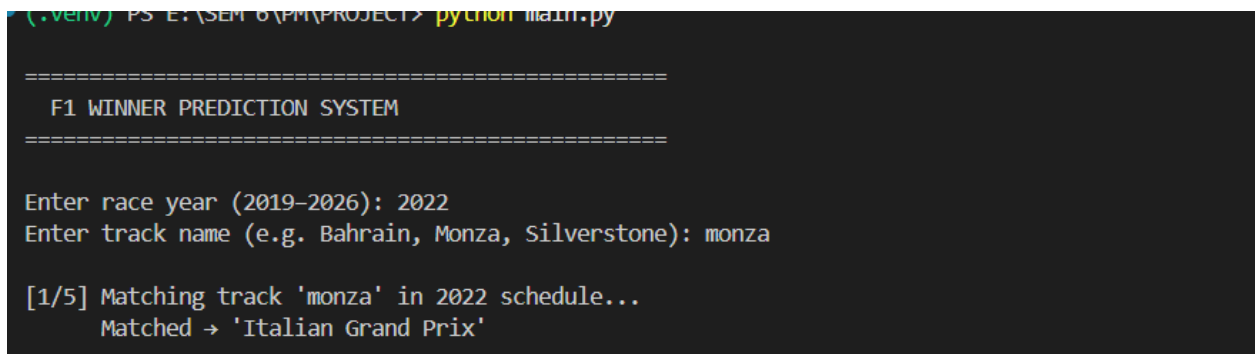
```
f1_project/ ├── main.py                                <- single entry point: python main.py ├──
f1_visualizer.py                                <- standalone demo: python f1_visualizer.py ├──
requirements.txt ├── data/ | └── cache/                <- FastFl parquet cache
(auto-created) └── src/ | └── __init__.py                <- empty - makes src a Python
package      ├── features.py | └── model.py            | └── simulation.py |
visualization/ | └── __init__.py                <- empty | └── f1_visualizer.py
<- pipeline-connected visualizer | └── track.py        <- track coordinate
loader
```

APPENDIX B — SCREENSHOTS

Insert the following screenshots after running the GridLogic system on your machine. Each placeholder box shows exactly which screenshot to add and where it belongs in the report.

B.1 Race Calendar Selection (Page 22 context)

Run `python main.py` and take a screenshot of the terminal showing the numbered race calendar list and the user typing a number to select a race.



```
(.venv) PS E:\SEM 6\PM\PROJECT> python main.py

=====
F1 WINNER PREDICTION SYSTEM
=====

Enter race year (2019-2026): 2022
Enter track name (e.g. Bahrain, Monza, Silverstone): monza

[1/5] Matching track 'monza' in 2022 schedule...
      Matched -> 'Italian Grand Prix'
```

B.1 — Terminal: Numbered race calendar list + user selecting race by number

B.2 Model Training Output (Page 26 context)

Screenshot the terminal output after training completes. It should show best params printed by GridSearchCV, the MAE and RMSE values, and the feature importance ranked table below.

```
[3/5] Building features...
Features built: 4651 laps, 12 features
Features ready: 4651 rows x 12 columns.

[4/5] Training XGBoost model...
Train samples: 3720 | Test samples: 931
Best params: {'colsample_bytree': 0.8, 'learning_rate': 0.05, 'max_depth': 4, 'n_estimators': 400, 'subsample': 0.8}
MAE on test set: 0.274 seconds

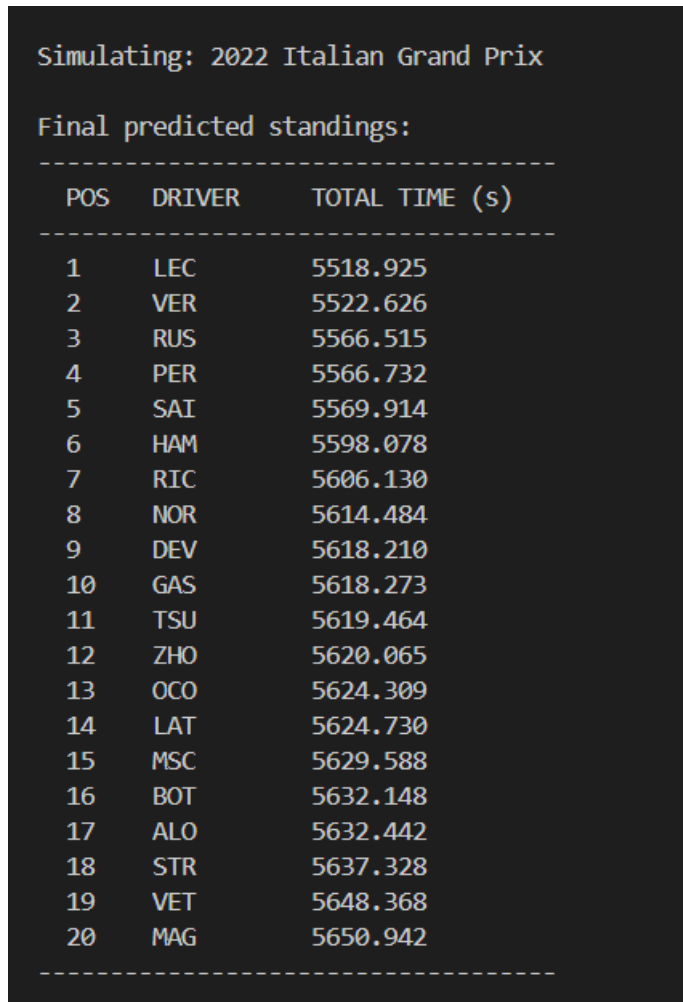
Feature importance:
sector1_time      0.6506
sector3_time      0.1436
sector2_time      0.1396
tyre_age          0.0204
lap_number        0.0178
fuel_load_est     0.0095
compound_code     0.0065
avg_lap           0.0034
best_lap          0.0031
team_strength     0.0030
driver_skill      0.0023
qual_pos          0.0000

Model training completed successfully ✓
```

B.2 — Terminal: GridSearchCV best params + MAE + feature importance table

B.3 Final Standings Console Output (Page 30 context)

Screenshot the terminal showing the predicted final standings table printed after the simulation completes. It should show position numbers, driver abbreviations, and total time values.



```
Simulating: 2022 Italian Grand Prix

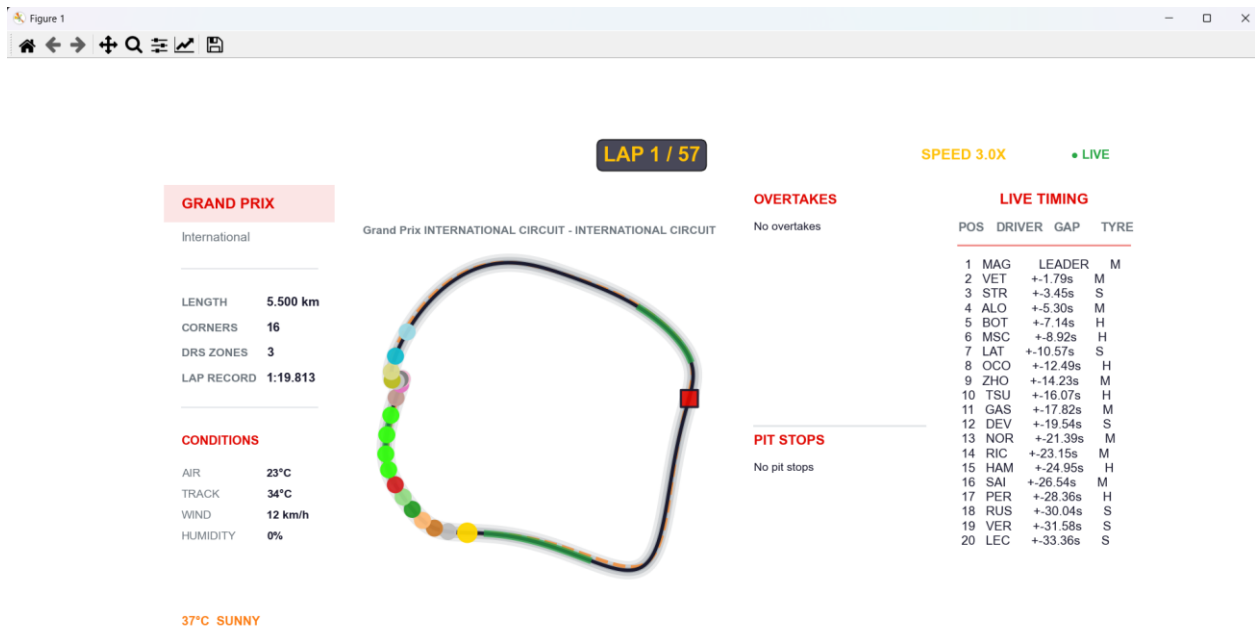
Final predicted standings:
-----
  POS  DRIVER    TOTAL TIME (s)
-----
   1    LEC      5518.925
   2    VER      5522.626
   3    RUS      5566.515
   4    PER      5566.732
   5    SAI      5569.914
   6    HAM      5598.078
   7    RIC      5606.130
   8    NOR      5614.484
   9    DEV      5618.210
  10    GAS      5618.273
  11    TSU      5619.464
  12    ZHO      5620.065
  13    OCO      5624.309
  14    LAT      5624.730
  15    MSC      5629.588
  16    BOT      5632.148
  17    ALO      5632.442
  18    STR      5637.328
  19    VET      5648.368
  20    MAG      5650.942
-----
```

POS	DRIVER	TOTAL TIME (s)
1	LEC	5518.925
2	VER	5522.626
3	RUS	5566.515
4	PER	5566.732
5	SAI	5569.914
6	HAM	5598.078
7	RIC	5606.130
8	NOR	5614.484
9	DEV	5618.210
10	GAS	5618.273
11	TSU	5619.464
12	ZHO	5620.065
13	OCO	5624.309
14	LAT	5624.730
15	MSC	5629.588
16	BOT	5632.148
17	ALO	5632.442
18	STR	5637.328
19	VET	5648.368
20	MAG	5650.942

B.3 — Terminal: Predicted final race standings table with all drivers

B.4 Visualization — Race Start

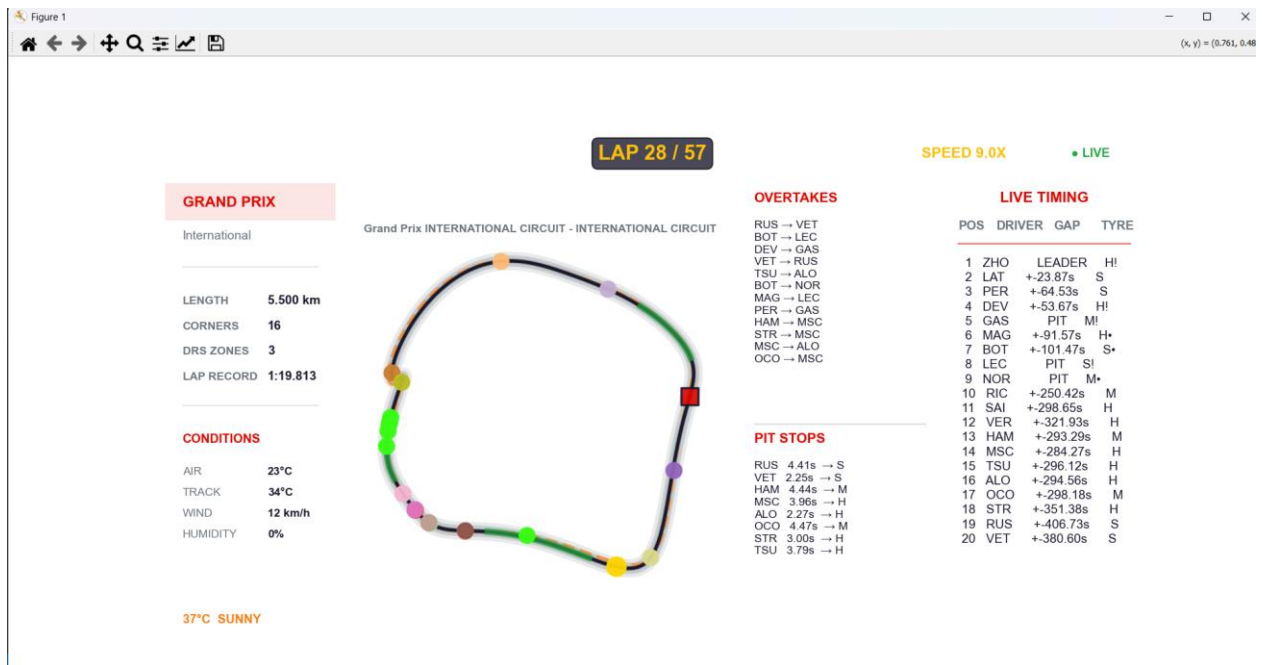
Run python f1_visualizer.py and take a screenshot of the full matplotlib window at race start. All 20 cars should be clustered near the start/finish line and the header should show LAP 1/57.



B.4 — matplotlib window: Race start with LAP 1/57 and all cars at start/finish

B.5 Visualization — Mid Race

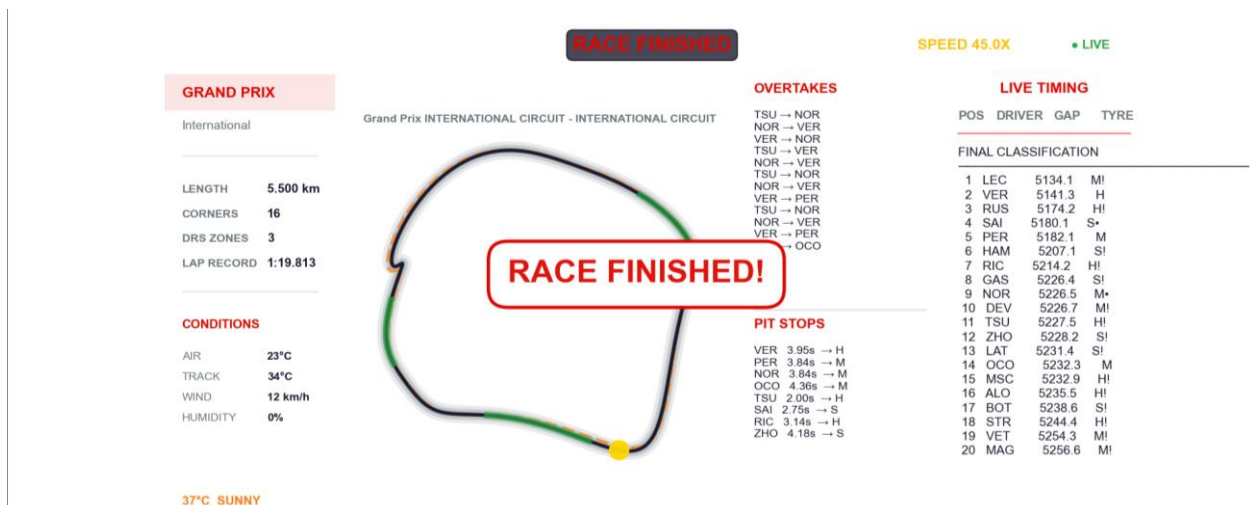
Let the simulation run to approximately lap 28 and take a screenshot. Cars should be spread around the circuit and the leaderboard on the right should show live gap values.



B.5 — matplotlib window: Mid-race with cars spread out and live leaderboard visible

B.6 Visualization — Race Finished

Let the simulation run to completion. Take a screenshot showing the green FINISHED text in the header and the frozen final leaderboard showing all positions.



B.6 — matplotlib window: Green FINISHED header and frozen final leaderboard